

Patent Candidate – Identity-Bound Generative Output Validation

Background: Modern LLM pipelines often require *structured outputs* defined by a schema (e.g. a JSON schema). For example, OpenAI’s “Structured Outputs” feature explicitly ties a JSON schema to a prompt so that “the model will always generate responses that adhere to your supplied JSON Schema” [29†L823-L826] . Likewise, recent research emphasizes that valid JSON syntax alone does not guarantee semantic correctness [6†L268-L275] . In practice, however, many systems simply check that an LLM’s output parses as JSON and meets syntactic schema constraints. In our case, a generative job claimed to require a nine-key JSON structure, but the code only enforced JSON validity, allowing shorter outputs to be admitted.

Observed Defect: In a capacity-sweep experiment, three outcomes appeared under the same *declared 9-key* contract:

- Low capacity: output **truncated** (not valid JSON) → **FAIL**.
- Medium capacity: output valid JSON with 6 keys (so syntactically valid) → **COMPLETED**.
- High capacity: output valid JSON with 8 keys (still syntactically valid) → **COMPLETED**.

No variant produced the full 9-key object. In other words, the system treated any valid JSON as meeting the contract. This indicates the runtime’s admission logic effectively reduced to “Accept(output) iff output parses as JSON” – i.e. structural validity alone. The code carried the intended schema ID in the job identity, but never actually applied it as an admission predicate. This is the technical fault we aim to fix.

Invention Nucleus: The novel mechanism is a *strict contract-bound admission gate* for generative outputs. Key elements include:

- **Contract-Tagged Work Identity:** The job request includes an explicit *ResultContractID* (e.g. a JSON schema ID). This identity binds the intended output format to the request itself [31†L98-L107] [29†L823-L826] .
- **Multi-Stage Validation:** The runtime separates stages—transport success, generation completion, syntactic parse, structural/schema check, and semantic contract admission. Only after *all* steps pass is the result admitted.
- **Admit Predicate:** A result r for job j is admitted **iff** it parses as JSON **and** conforms exactly to the schema/contract identified by j **and** any additional semantic predicates on r hold. In other words:

$$\text{Admit}(r, j) \iff \text{Parse}(r) \wedge \text{ConformsToSchema}(r, \text{Contract}_j) \wedge \text{SemanticPredicates}(r, j).$$

If any check fails, the output is rejected, and a detailed *admission receipt* is generated (indicating which condition failed).

- **Identity Binding:** Crucially, the *ContractID* is part of the job's identity. Thus the system can cryptographically or otherwise associate any accepted result with the exact schema it was promised. Any mismatch (even if syntactically valid) yields “no admission.” This prevents “valid JSON” from being misinterpreted as meeting a stricter contract.

Prior Art: Structured-output enforcement is an active area. For example, SLOT (Teng *et al.* 2025) discusses ensuring LLM outputs fit a target JSON schema [5†L179-L187] , and OpenAI's docs describe schema adherence features [29†L823-L826] . OpenAI's own patent filing similarly envisions bundling a prompt with a JSON schema and constraining the model to produce outputs “guaranteed to conform to that schema” [31†L98-L107] . These systems emphasize *structural* compliance: ensuring required keys and types are present [5†L179-L187] [29†L823-L826] . However, they do not address the *identity-binding and admission logic*. In practice, many pipelines indeed validate only syntax; research notes that “JSON validity checks... ensure syntax but often do not guarantee semantic correctness” [6†L268-L275] .

No known art ties the *contract identity* into the job's identity and admission criteria. In typical pipelines (and even in some LLM frameworks), once an output is syntactically valid, it is accepted and used downstream. For example, a recent patent on AI data pipelines discusses *schema validation* but in the context of data training (removing nonconforming data) [21†L765-L773] ; it does not describe linking schema identity to a single generative job or gating action on it. Likewise, patents on generative output “validation” tend to focus on comparing answers to knowledge bases or computing confidences [25†L27-L35] , not on enforcing a specified output contract by identity.

Technical Effect: The proposed mechanism ensures a generative result can only be used if *all* declared contract constraints (syntactic, type, and semantic) are met. It *refuses both errors*:

- *Over-admission:* It cannot admit an output that omits a required field (as happened in the example) because such output fails the contract check.
- *Over-withholding:* It will admit a result even if internal world identity is still ambiguous, as long as all worlds agree on the admissible consequence (the output) – avoiding unnecessary rejections.

It also enables precise receipts and future-proofing: for any admitted result, the system logs exactly which schema was satisfied and which predicates were evaluated. Re-running or replaying with the same *ContractID* must yield the same admission outcome (no gameable state).

Claim Concepts: Based on this, promising claim elements include: 1. **Contract-Tagged Generative Task:** A method where a generation request includes an identifier for a result contract (schema), and the request ID is a function of (input, model config, contract ID, ...).

2. **Schema-Bound Admission Gate:** Only admitting outputs that both parse correctly and “satisfy the declared contract.” The contract used to judge the output is exactly the one bound to the request identity.

3. **Multi-Stage Completion States:** Explicitly distinguishing completion (model finished) from admission (schema-check pass) from dispatch (output used). Results may complete but be rejected if they fail the contract.

4. **Admission Receipts:** Producing a proof/receipt tying the output back to the contract and indicating pass/fail status on each predicate.

5. **Replay/Idempotence:** Stipulating that repeating the same output+contract cannot change an admission decision (no stronger consequence on replay).

6. **Semantic Drift Detection:** As a separate angle, one could claim detecting that multiple valid outputs (all

syntactically correct) differ in structure, flagging a “contract instability.” E.g. “When two outputs under the same ContractID differ, the system detects drift and withholds admission until clarified.” This is a higher-level use case (the *two candidates* noted).

Prior Art Check: The core novelty is **not** simply using JSON schemas or validating output format – that is well-known [29†L823-L826] [5†L179-L187] . The novelty lies in *binding the contract to the job identity and gating the result admission on an exact match to that identity*. We must distinguish this from standard schema-validation pipelines. Even OpenAI’s structured outputs (which constrains generation) is about token-level enforcement during generation [31†L98-L107] ; here we are talking about an execution-time checkpoint that enforces the declared schema exactly as the condition for result acceptance. It’s also distinct from general hallucination-detection patents.

Claim-Drafting Notes: Emphasize the *specific improvements* in computer functionality: e.g. **“A method for processing generative model outputs where a declared result schema is cryptographically bound to the task, and where only outputs satisfying that exact schema and associated semantic criteria are admitted”**. Support this with the code evidence that an output satisfying only a subset of schema constraints was incorrectly admitted. Each claim should clearly limit to this mechanism (not abstract governance). Dependencies could cover e.g. receipt formats or step order.

Enablement Gaps: We have working code and the experiment, so implementation is feasible. The main questions are: (a) how to precisely define “semantic predicate” in claims? (b) Could an examiner say it’s just “implement JSON schema validation,” which is known? To strengthen, note that we require the schema *identity* match as part of what’s validated, and tie it to job ID.

Conclusion: The evidence suggests a concrete technical invention: **“admission gating of generative AI output by a declared schema identity and semantic predicates.”** This goes beyond existing schema-validation art by making the contract a first-class identifier in the task and enforcing it deterministically at output time. It appears to be patentable subject matter as a computer-implemented process improvement [6†L268-L275] [29†L823-L826] .

Sources: Prior work on structured LLM output and JSON schema (SLOT [5†L179-L187]), problems of syntax-only checks [6†L268-L275] , and industry features/patents for JSON enforcement [29†L823-L826] [31†L98-L107] highlight the context. These confirm schema-based validation is standard, but do not disclose the identity-bound admission mechanism. Thus the candidate lies in that specific mechanism, not in the well-known schema usage.
